

A · P · U

ASIA PACIFIC UNIVERSITY
OF TECHNOLOGY & INNOVATION

Module Code	**:**	CT080-3-2 (Ethical Hacking and Incident Response)
Intake Code	**:**	APU2F2602CS(CYB)
Submission Title	**:**	Assessment of LLMNR/NBT-NS Poisoning for Credential Capture, Offline Password Cracking, and Unauthorized Access to Simulated Healthcare Data in an Isolated Windows Testbed
Lecturer Name	**:**	Ts. Mohd Norfahmi bin Jaafar & Zaireeda binti Mohd Fauzee
Hand in Date	**:**	-
Group No.	**:**	V
Group Leader	**:**	Swann Paing Sett

Student TP Number	**Student Name**
TP096026	Swann Paing Sett

Contents

References..... 4

1. Introduction

This report covers Section 1 of the assignment: an individual ethical hacking demonstration. The chosen domain is Password Attack and Authentication Testing. The full title of the work is "Assessment of LLMNR/NBT-NS Poisoning for Credential Capture, Offline Password Cracking, and Unauthorized Access to Simulated Healthcare Data in an Isolated Windows Testbed". The target industry is Healthcare.

Hospitals keep patient records on internal file servers. Staff log in to these servers every day using Windows accounts. If an attacker can steal those logins, they can read private patient data. This breaks the law and harms patients. So it is important to test how safe these logins really are.

The attack shown here does not guess or brute-force a password at the login screen. Instead, it tricks Windows name services (LLMNR and NBT-NS) so that a victim computer sends its authentication response – a challenge-response value that contains a hashed form of the password, not the password itself – to the attacker by mistake. The attacker then cracks that response offline to recover the password, or, where the conditions allow, relays it to another server. This class of attack is listed by MITRE ATT&CK as technique T1557.001 (LLMNR/NBT-NS Poisoning and SMB Relay). The whole test was done in a closed lab with no internet-facing systems and with made-up patient data, so no real person or real network was harmed.

Credential theft is one of the most common ways real breaches begin: stolen logins, not zero-day bugs, are the usual entry point. LLMNR/NBT-NS poisoning is a favourite early move because the two protocols are on by default on Windows and most networks never turn them off, which makes it realistic for a hospital setting.

Every screenshot shows the student's own account (swann), as required, to prove the work is individual.

2. Scenario and Target Environment

The scenario is a small hospital network. One Windows Server 2012 machine acts as the hospital file server. It holds a shared folder called Patient_Records that stores patient files, and

staff computers on the same LAN connect to it to open those files – a very common setup in real clinics.

The attacker is an insider or someone who has already reached the internal network (for example through a phishing email or an open network port). The attacker has a laptop running Kali Linux on the same LAN. The attacker has no password and no admin rights at the start. The goal is to get a valid staff login and use it to read patient records.

This scenario is believable because hospital networks grow over time: old machines stay running for legacy software, name services like LLMNR stay on because no one turns them off, and file servers hold years of patient data in simple shared folders. The attack below takes advantage of these everyday defaults, so the test mirrors a real risk rather than an artificial one.

The test used an isolated host-only network (192.168.40.0/24) in VMware Workstation, so no traffic ever left the host PC. One Kali attacker and three Windows machines were used. Table 1 lists every machine, its address, its software version and what it does in the test.

Machine (role)	**Operating system (version)**	**IP address**	**Key tools / service**	**Role in the test**
Kali Linux (attacker)	Kali Linux (rolling)	192.168.40.128	Responder 3.2.2.0, Impacket (ntlmrelayx), hashcat 7.1.2, Nmap 7.991, smbclient	Runs the poisoning, hash capture, cracking and file access
Windows Server 2012 (target – file server)	Windows Server 2012 (6.2.9200)	192.168.40.130	SMB file sharing; Patient_Records share	Hospital file server that holds the patient data
Windows XP (victim client)	Windows XP Professional (5.1.2600)	192.168.40.131	SMB / WebDAV client	Staff client whose login is poisoned, captured and cracked
Windows 2000 (victim client – tested)	Windows 2000 Professional (5.0.2195)	192.168.40.129	Legacy SMB1 client	Tried first, then set aside (legacy port 139 / SMB1 limits)

Table 1: Lab environment – machines, addresses, versions and tools.



Figure 2.1: *Lab environment*

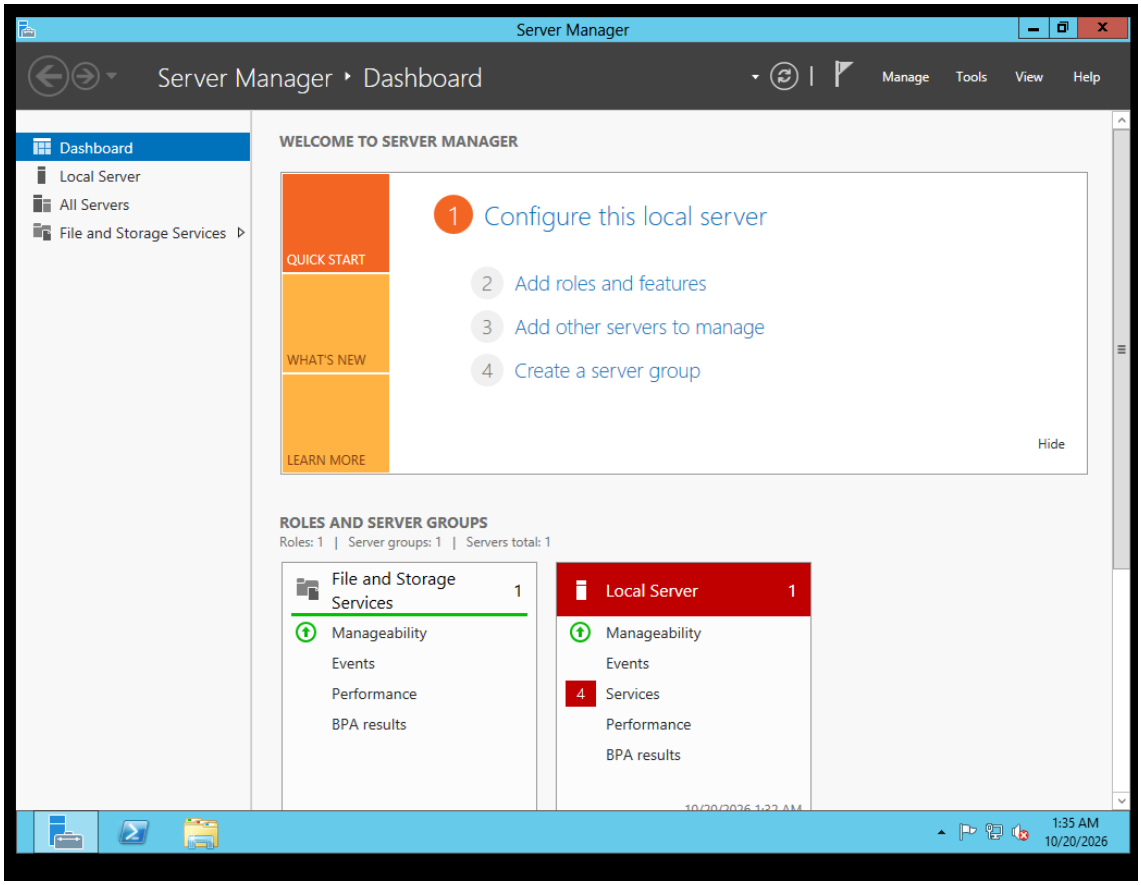


Figure 2.2: *Windows Server file server*

3. Attack Technique, Tools and Framework

This section explains the main idea of the attack and the tools used. Understanding the “why” makes the later steps easy to follow.

3.1 How the attack works

When a Windows computer wants to reach a server by name (for example \\FILESERVER01), it first asks the DNS server. If DNS has no answer, Windows falls back to two old broadcast protocols: LLMNR (Link-Local Multicast Name Resolution) and NBT-NS (NetBIOS Name Service). With these, the computer simply shouts to the whole local network: “who is FILESERVER01?” Any machine can answer.

This is the weakness. The attacker runs a tool that answers every such shout and says “I am that server”. The victim believes it and tries to log in to the attacker’s machine. During that login, Windows sends an NTLM authentication message that contains a hashed form of the user’s password. The attacker now holds that hash. Two things can then be done with it:

- Offline cracking – feed the captured hash to a cracking tool with a wordlist and recover the real plaintext password.

- NTLM relay – pass the login straight on to a real server before it expires, and log in there as the victim without ever knowing the password.

NTLM comes in two versions. NTLMv2 is newer and harder to crack; NTLMv1 is older and much weaker, which is why legacy clients like Windows XP help an attacker. Both use a challenge-response: the server sends a random challenge and the client answers with a value made from the challenge and the password hash. Responder plays the server and sets the challenge, so the answer it receives can be cracked or relayed.

3.2 Tools used

Responder – an open-source tool that poisons LLMNR, NBT-NS and MDNS. It pretends to be the server the victim is looking for, captures the NTLM hash and saves it to a log file.

Impacket (ntlmrelayx) – a set of Python network tools. The ntlmrelayx script takes the poisoned login and relays it to a chosen target server to gain access without cracking anything.

Nmap – a scanner used to find open ports and to check the SMB security settings of the target server.

hashcat – a fast password-cracking tool. It was used in NetNTLM mode to turn the captured hash back into the plaintext password using the rockyou wordlist.

smbclient – a Linux SMB client used at the end to log in to the server with the cracked password and download the patient files, proving real impact.

All five tools are free and open-source, and using focused tools rather than one all-in-one framework keeps each step clear and easy to explain.

The work follows the standard ethical hacking methodology taught in the module – Footprinting, Scanning, Vulnerability Identification, Exploitation, Impact, and Countermeasures – shown below in order.

4. Step-by-Step Attack Demonstration

4.1 Phase 1 – Footprinting and Reconnaissance

The first job is to learn the layout of the network: which machine is the attacker and which is the target. On the Windows Server the command ipconfig shows the server's IP address is 192.168.40.130. This is the hospital file server that will be attacked.

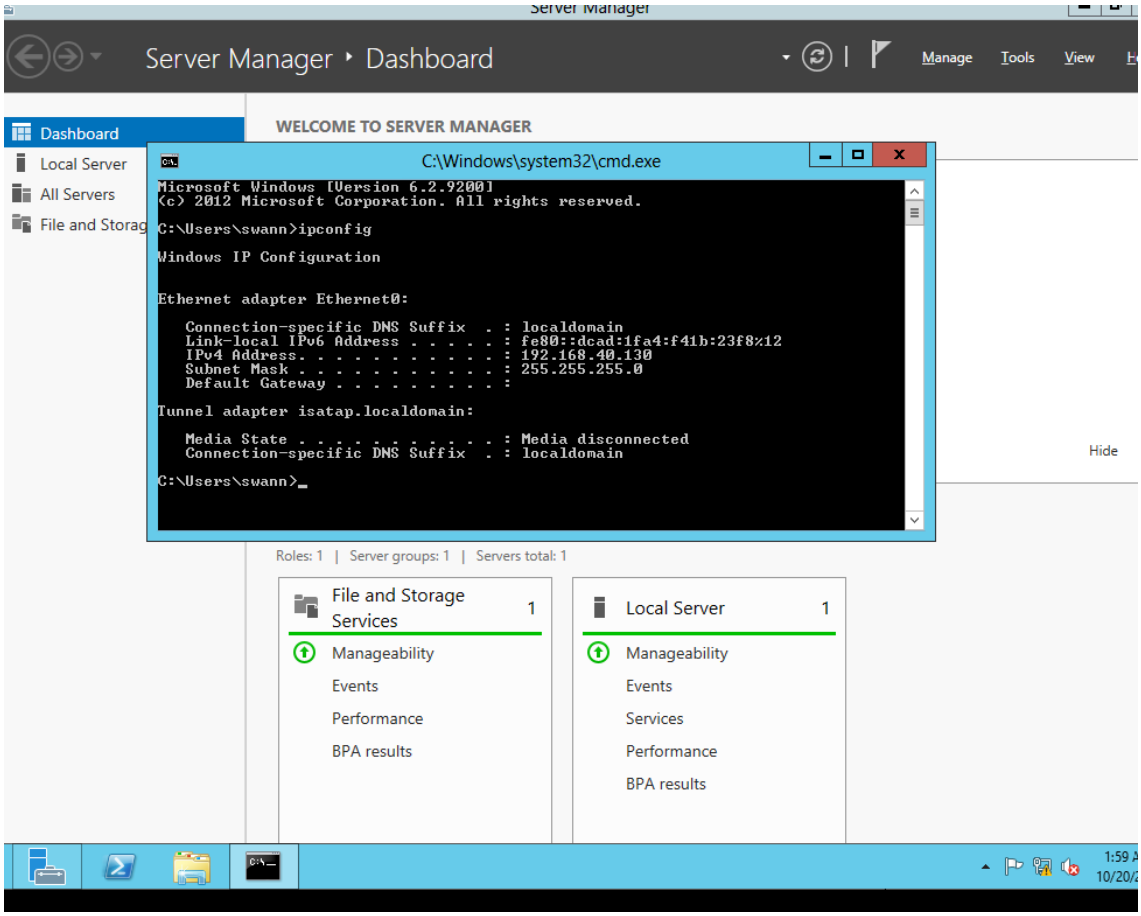


Figure 4.1.1: *Target server IP*

On Kali the command ip a shows the attacker's own IP address is 192.168.40.128 on interface eth0. Both machines sit in the same 192.168.40.0/24 network, so the attacker can reach the

server. No noisy scanning is needed yet – this step is passive and simply confirms the environment.

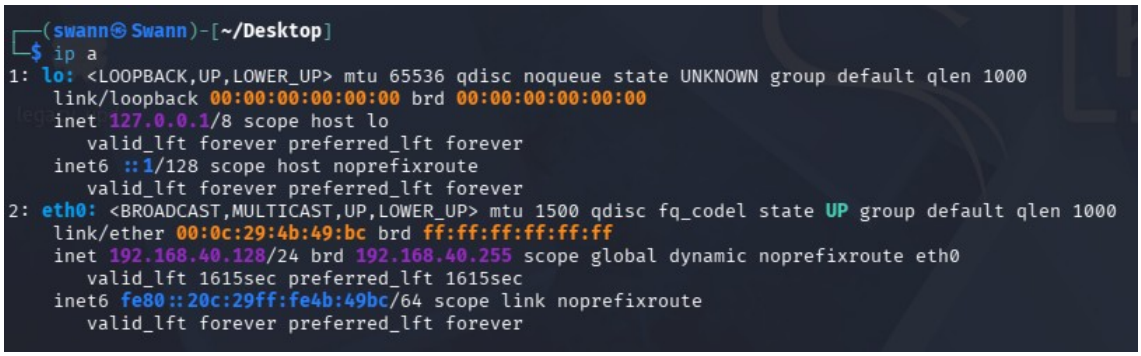


Figure 4.1.2: *Kali attacker IP*

The server runs the File and Printer Sharing feature, which is what lets staff reach shared folders over SMB. This feature is turned on, confirming the server behaves like a real file server.

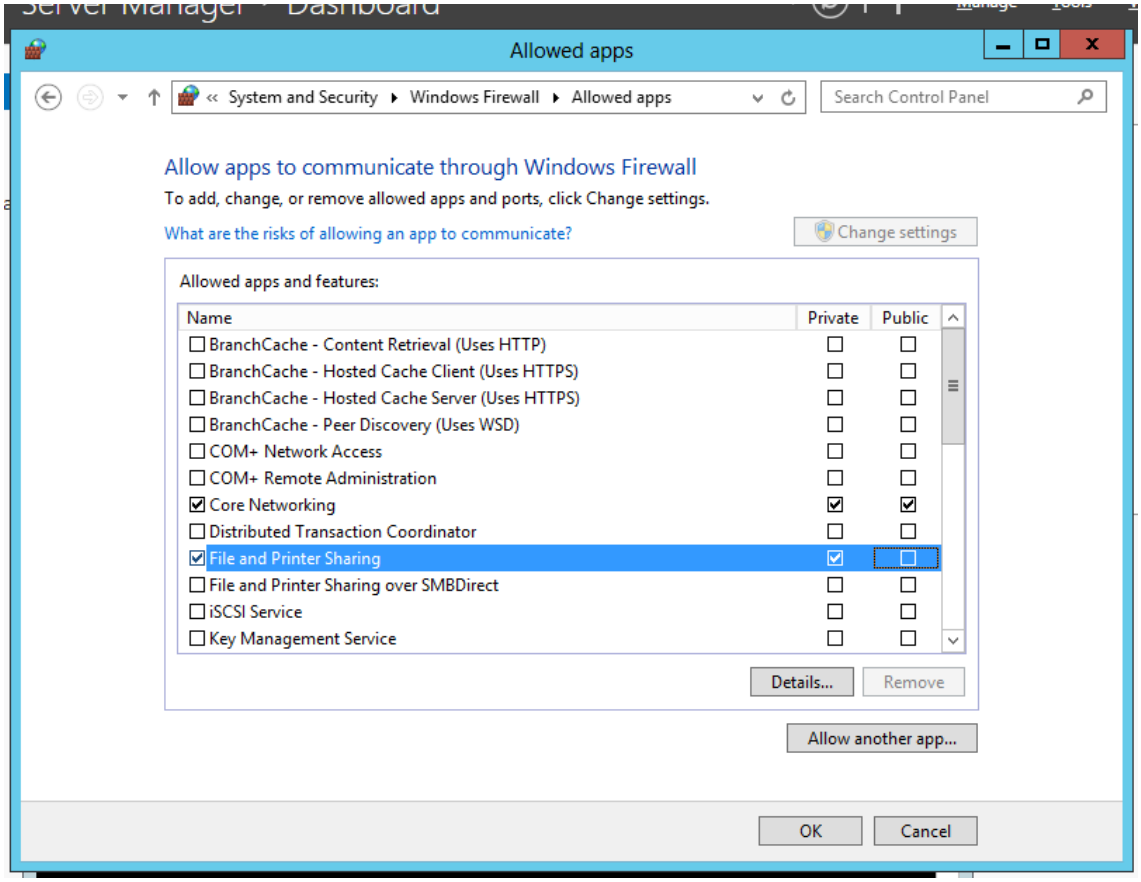


Figure 4.1.3: *File and Printer Sharing enabled*

4.2 Phase 2 – Scanning and Enumeration

Before scanning, the attacker runs a quick connectivity check with ICMP ping. From Kali, a ping to the Windows XP client (192.168.40.131) succeeds with 0% packet loss. This shows the attacker's own network link is working and that a host on this subnet can be reached normally.

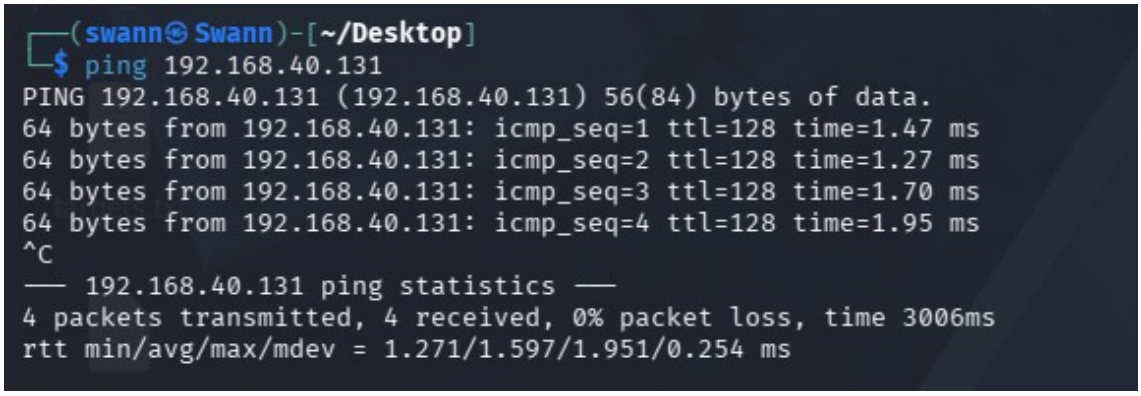


Figure 4.2.1: *Kali can ping the client (connectivity OK)*

However, a ping from the same Kali machine to the target server (192.168.40.130) returns 100% packet loss – every request times out. Because Kali can reach the client but not the server, the problem is not the attacker's network; it is the server's own firewall dropping inbound ICMP.

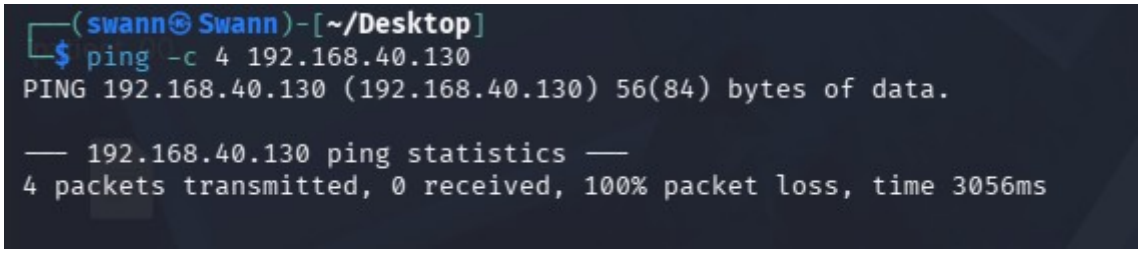


Figure 4.2.2: *Kali cannot ping the server (100% loss)*

This is confirmed by pinging in the other direction: the server itself can ping the Kali machine (192.168.40.128) successfully. So the server has normal outbound connectivity, and only inbound ICMP is blocked by its firewall. This matters for scanning, because a normal scan that first pings the host would wrongly mark the server as down.

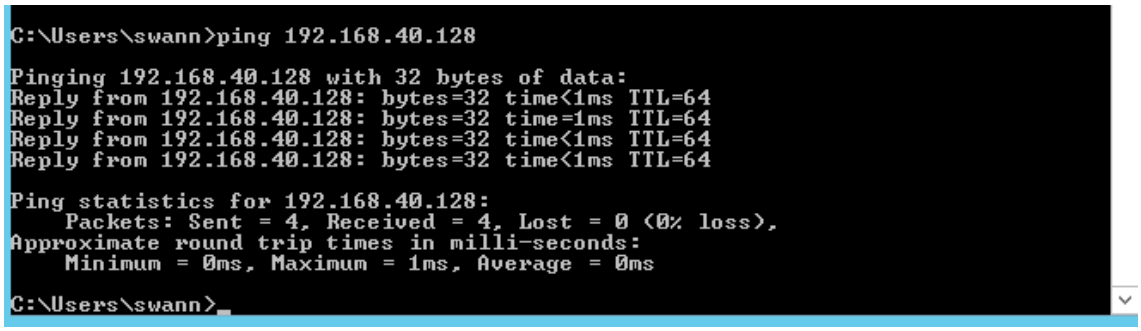


Figure 4.2.3: *Server can ping out (block is inbound only)*

For this reason the attacker scans the server with nmap using the -Pn option, which skips host discovery (the ping check) and tests the TCP port directly. As Figure 4.2.4 shows, SMB port 445 is open (microsoft-ds) and the host is a VMware machine. An open SMB port means the

file-sharing service is reachable and is a good attack target. The same scan can also report the server's SMB security settings, which are examined as the key vulnerability in the next phase.

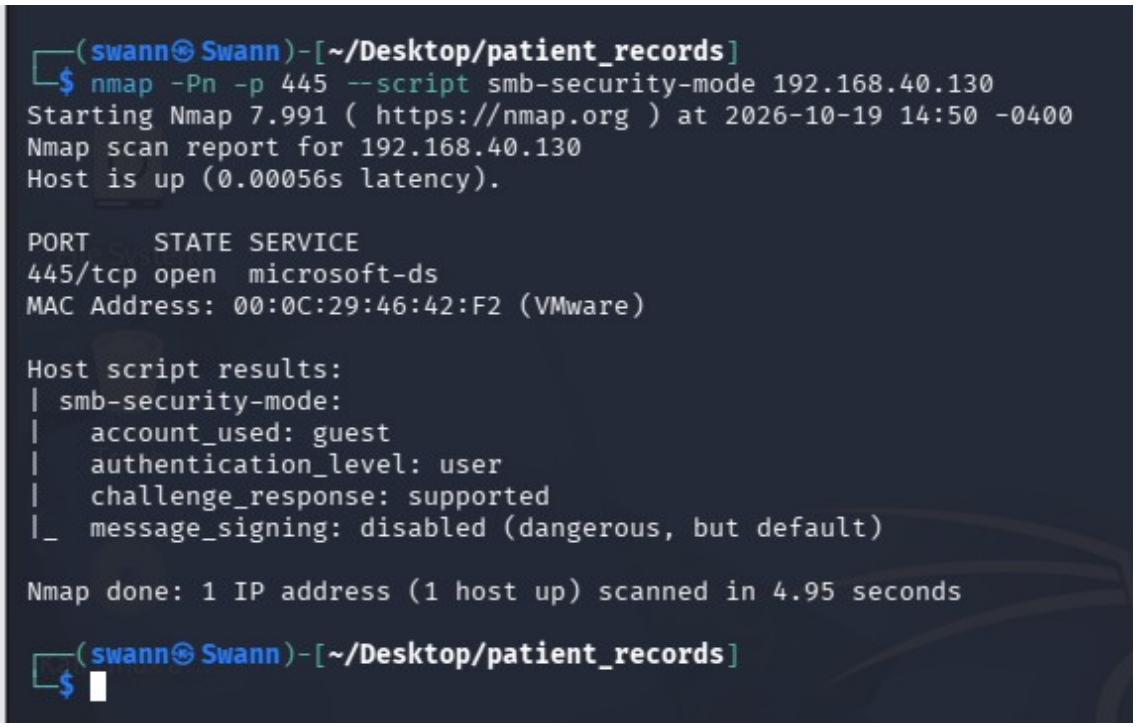


Figure 4.2.4: *Nmap scan – SMB port 445 open*

To make the scenario realistic, two sample patient files (patient_001 and patient_002) were placed in a folder and that folder was shared as Patient_Records. These files contain made-up data only – no real patient information. This shared folder is the sensitive asset the attacker wants to reach.

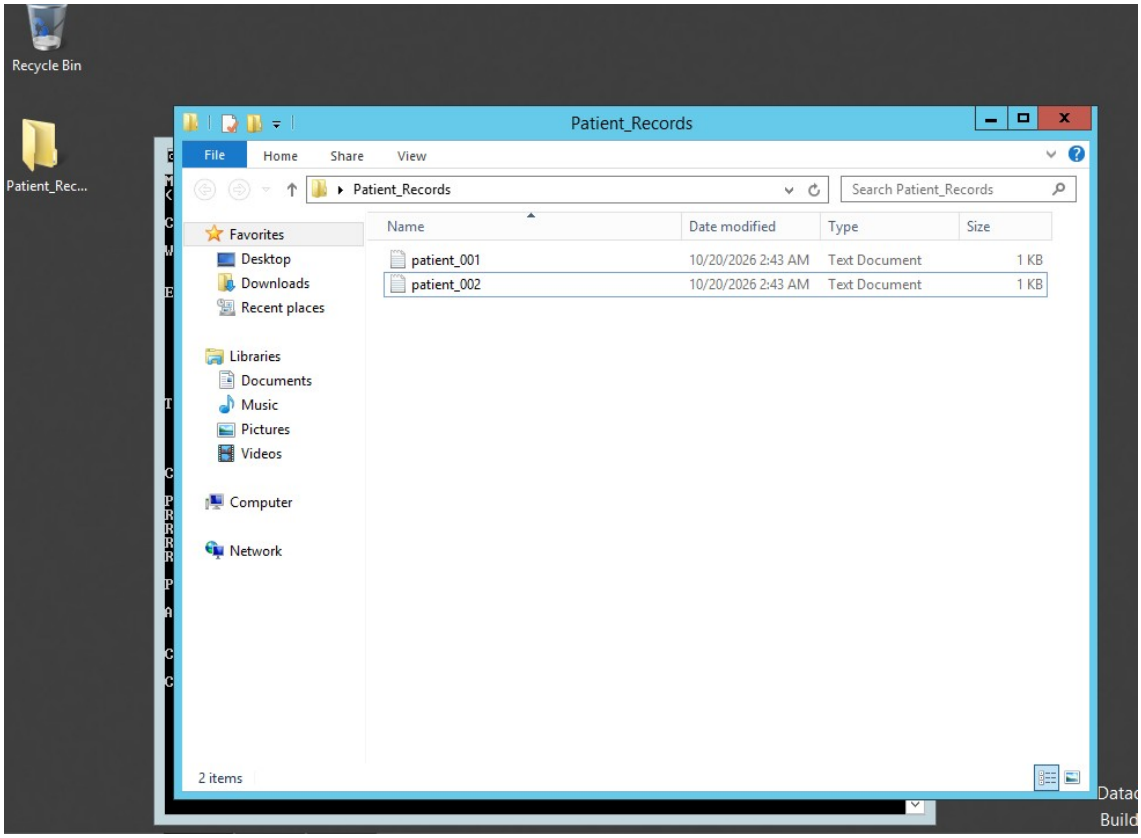


Figure 4.2.5: *Patient_Records folder*

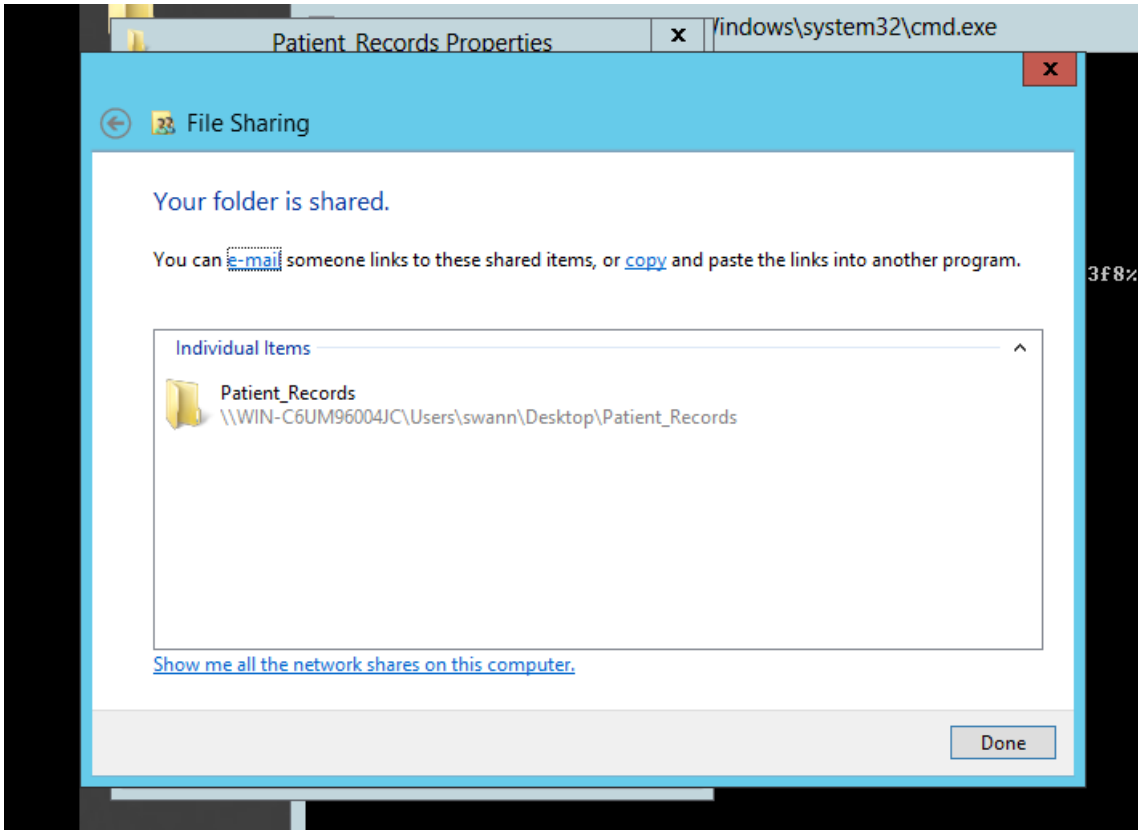


Figure 4.2.6: *Folder shared on network*

4.3 Phase 3 – Vulnerability Identification

A focused scan with the smb-security-mode and smb2-security-mode scripts identifies the real weakness (Figure 4.3.1). The line message_signing: disabled (dangerous, but default), together with the SMB2 result “Message signing enabled but not required”, means the server does not force SMB signing. SMB signing is a check that proves a login message was not changed or replayed by a third party. When it is off, an attacker may be able to relay a captured login to the server. On a non-domain Windows Server this signing-disabled state is the default. It is a necessary condition for an NTLM relay attack, though not a guarantee: a relay may succeed only when the other required conditions are also met, such as a separate victim and target host and compatible SMB versions.

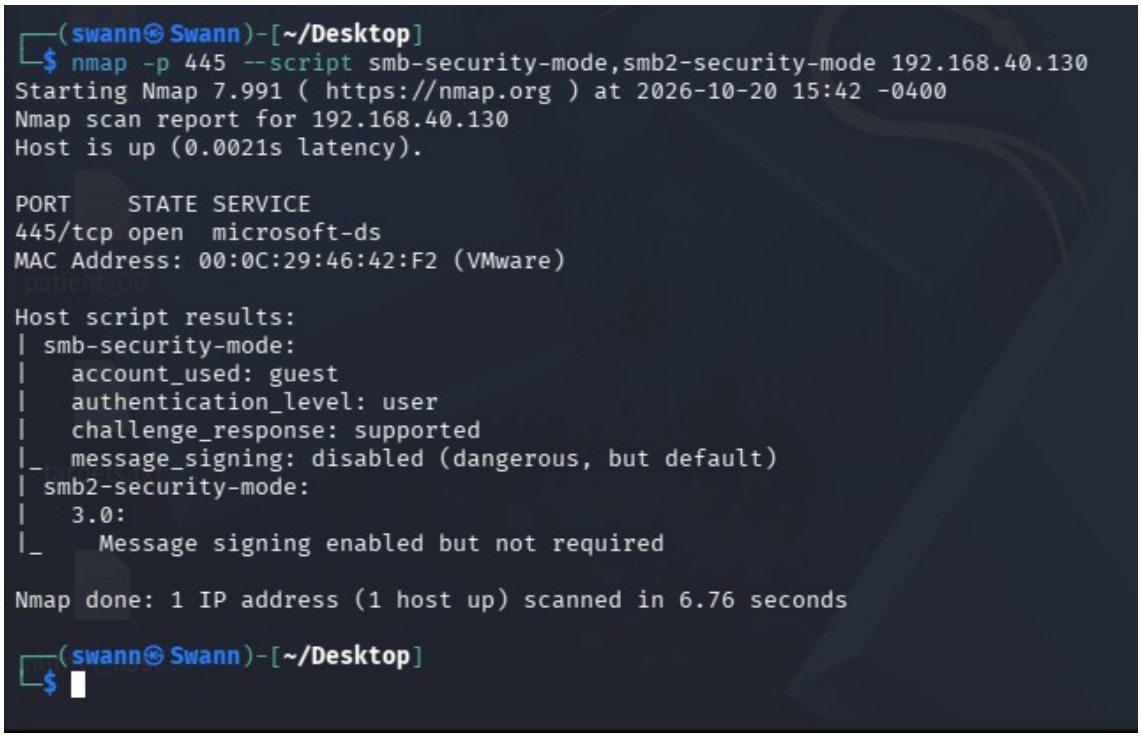


Figure 4.3.1: *Nmap (smb + smb2 scripts) – SMB signing disabled (the vulnerability)*

So the target has two linked problems: (1) LLMNR/NBT-NS are on, which lets the attacker steal logins, and (2) SMB signing is off, which may allow stolen logins to be relayed to the server when the other relay conditions are met.

4.4 Phase 4 – Exploitation (Attack Demonstration)

This is the main attack. Responder is started on Kali to poison the name-service requests. The startup screen shows the poisoners LLMNR, NBT-NS and MDNS are ON, and the tool is listening on the attacker’s interface.

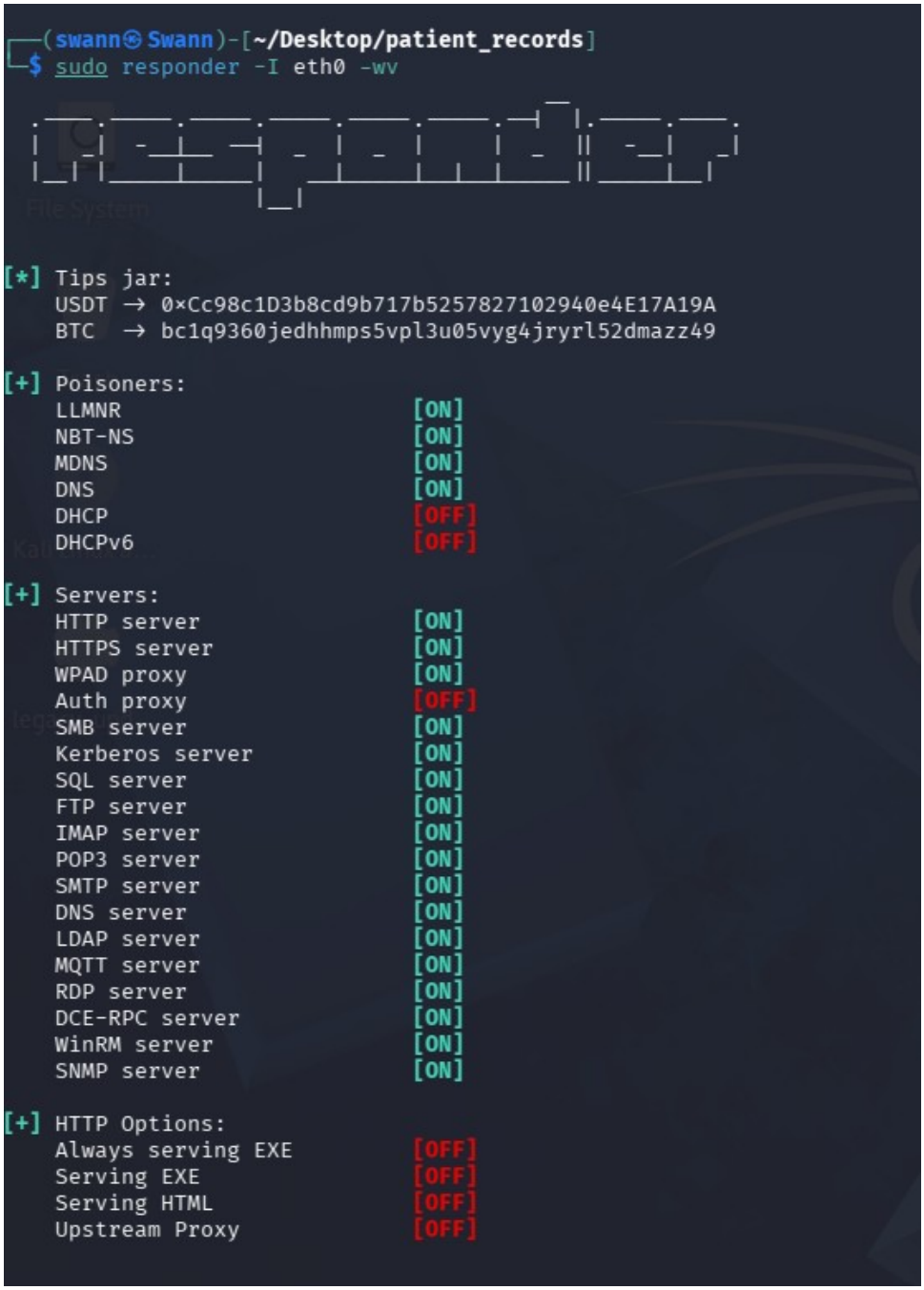


Figure 4.4.1: *Responder started*

To make a victim send its login, a Windows machine must try to reach a server by a name that DNS cannot resolve. In the lab this is triggered on the client by running net use \\ FILESERVER01\SharedDocs. The command fails with “System error 53 – the network path was not found”, because FILESERVER01 does not exist. But the failure comes too late: before giving up, Windows has already broadcast the name over LLMNR/NBT-NS, which is exactly what Responder is waiting for.

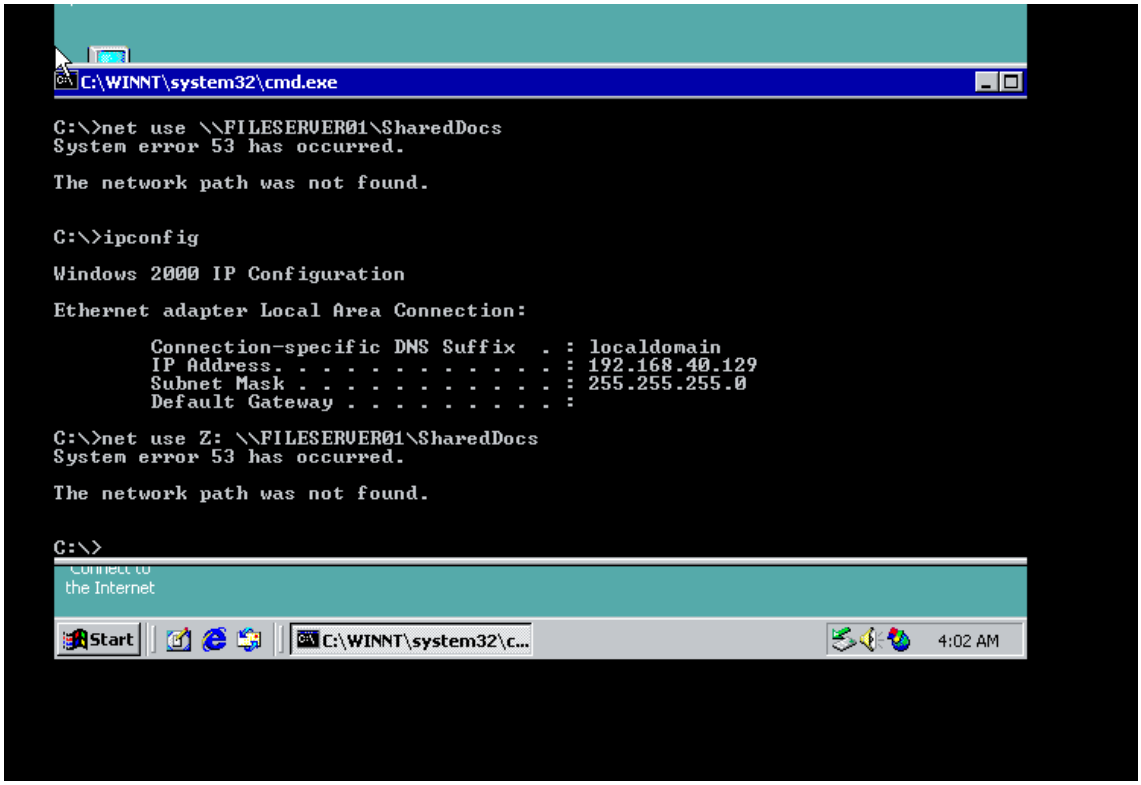


Figure 4.4.2: *Victim triggers the lookup (net use to a fake server name)*

It is worth being clear about why this is realistic. In a real network no user ever types this command. The same lookup happens automatically all the time – from Web Proxy Auto-Discovery (WPAD) requests browsers make on their own, from mapped drives that still point at decommissioned servers and retry at every logon, from logon scripts, and from simple typos in a share name. The manual command here only reproduces, on demand, what a production network produces by itself. In fact the NTLMv1 hash cracked later was captured through Responder's automatic WPAD/WebDAV path, not a typed command – a fully automatic, no-user-interaction trigger.

Once a Windows machine looks for a server name that DNS cannot answer, Responder sends a poisoned reply and the victim connects to the attacker. The log shows poisoned answers being sent and an SMB NTLM login being captured for the account swann, including the NetNTLMv2 challenge-response value (commonly called the NTLMv2 hash). This value is not the plaintext password; it must still be cracked offline.



Figure 4.4.3: *Responder LLMNR poisoning*

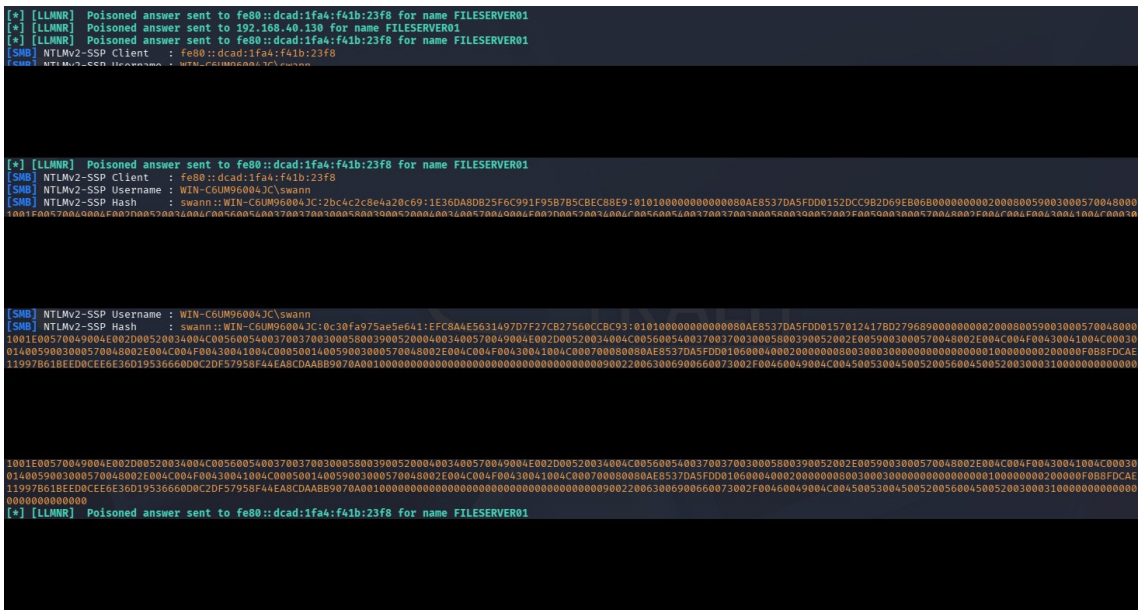


Figure 4.4.4: *NTLMv2 hash captured*

The captured hash is written to a log file on Kali. Using find and cat, the NTLMv2 hash for the swann account is located and read back from the Responder log.



Figure 4.4.5: *Finding the hash log file*

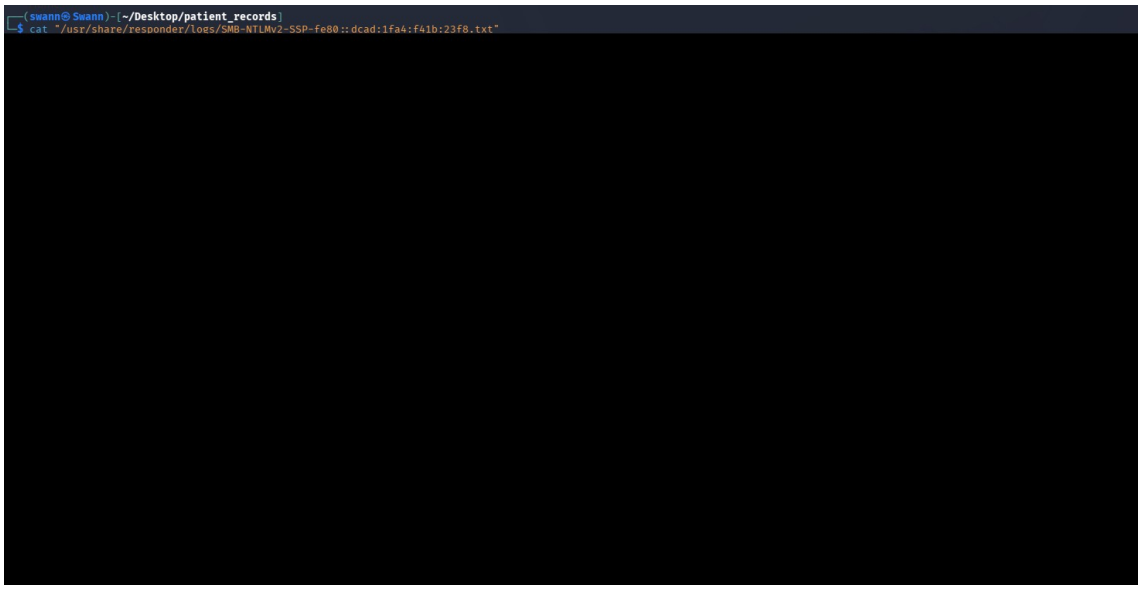


Figure 4.4.6: *Captured hash in log*

The next aim was to relay this login straight to the server with Impacket. Responder's own SMB server was first turned off in Responder.conf so it would not clash with the relay tool's own SMB server.

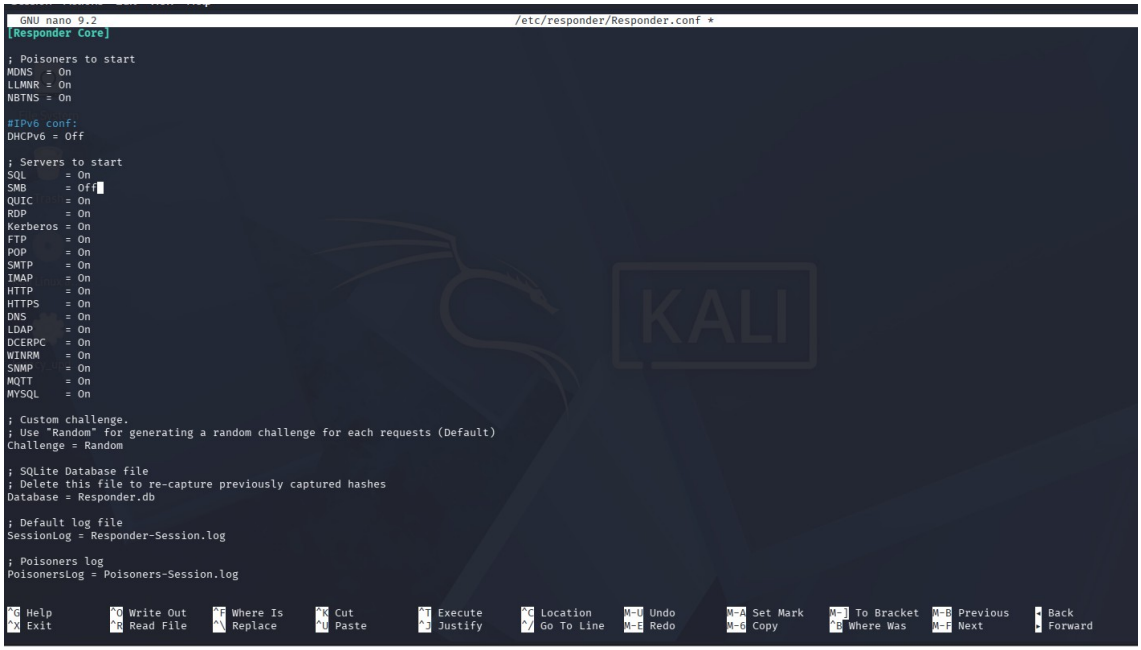


Figure 4.4.7: *Responder SMB turned off*

ntlmrelayx was then started with the server's IP as the target. The first relay attempts did not succeed. When the attacker relayed the server's own login back to the same server, Windows blocked it with its built-in loopback / reflection protection, which stops a host's credentials being relayed back to itself. The tool also showed port and start-up errors while the two SMB servers competed for the same port. These are real troubleshooting steps, shown here to explain why a second victim machine was needed.

```
┌──(swann㉿Swann)-[~/…/share/Trash/files/patient_records]
└─$ cd ~/Desktop

┌──(swann㉿Swann)-[~/Desktop]
└─$ echo "192.168.40.130" > targets.txt

┌──(swann㉿Swann)-[~/Desktop]
└─$ impacket-ntlmrelayx -tf targets.txt -smb2support
Impacket v0.13.1 - Copyright Fortra, LLC and its affiliated companies

[*] Protocol Client MSSQL loaded..
[*] Protocol Client SMB loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client RPC loaded..
[*] Protocol Client DCSYNC loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client WINRMS loaded..
[*] Running in relay mode to hosts in targetfile
[*] Setting up SMB Server on port 445
[*] Setting up HTTP Server on port 80
Exception in thread Thread-2:
Traceback (most recent call last):
```

Figure 4.4.8: *ntlmrelayx relay attempt*

```
OSError: [Errno 98] Address already in use
  File "/usr/lib/python3.14/threading.py", line 1082, in _bootstrap_inner
    self._context.run(self.run)
  File "/usr/lib/python3/dist-packages/impacket/examples/ntlmrelayx/servers/rdprelayserver.py", line 417, in run
    self.server = self.RDPServer((self.config.interfaceIp, self.config.listeningPort), self.RDPHandler, self.config)
  File "/usr/lib/python3/dist-packages/impacket/examples/ntlmrelayx/servers/rdprelayserver.py", line 98, in __init__
    socketserver.TCPServer.__init__(self, server_address, RequestHandlerClass)
  File "/usr/lib/python3.14/socketserver.py", line 457, in __init__
    self.server_bind()
  File "/usr/lib/python3.14/socketserver.py", line 478, in server_bind
    self.socket.bind(self.server_address)
OSError: [Errno 98] Address already in use
```

Figure 4.4.9: *Relay blocked (self-relay)*

To get a clean victim that is different from the target, a separate client machine was brought online. A Windows 2000 client was tried first (IP 192.168.40.129), but because it uses the very old NetBIOS port 139 rather than port 445, the relay tool did not pick up its login. The test then moved to a Windows XP client instead.

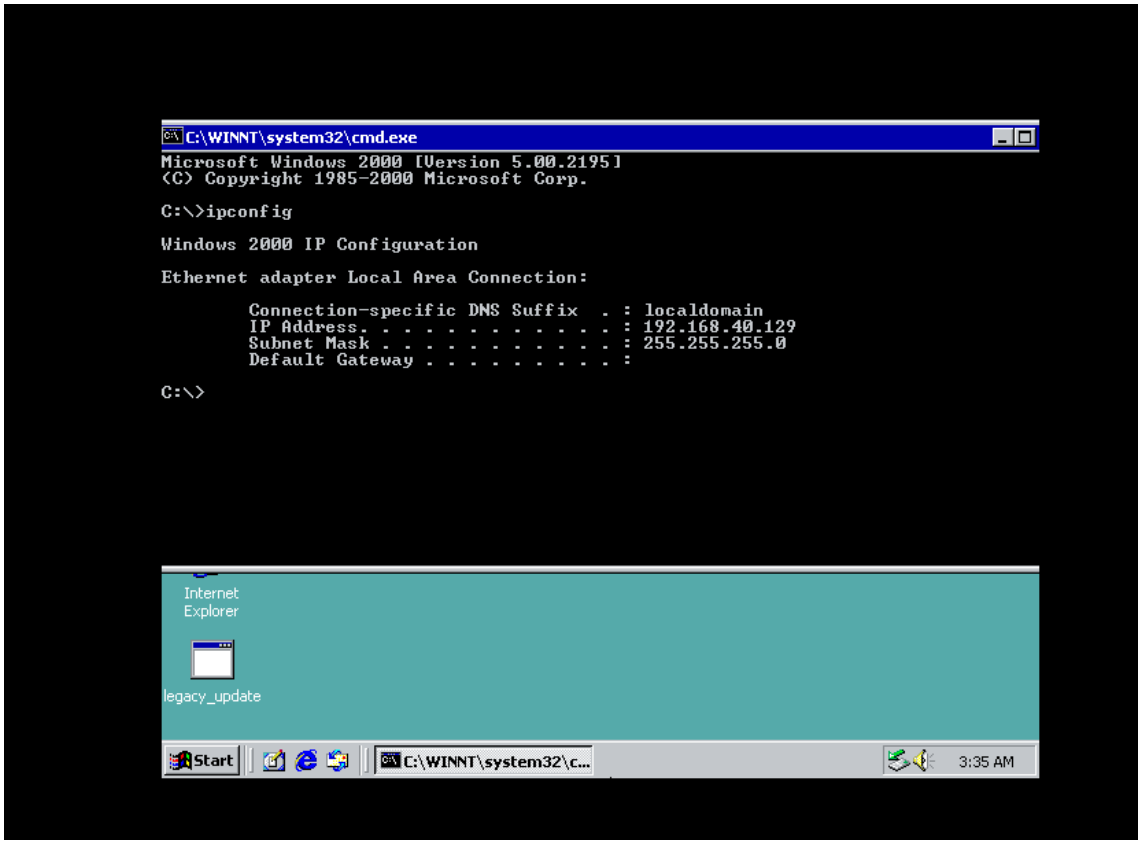


Figure 4.4.10: *Windows 2000 client IP*

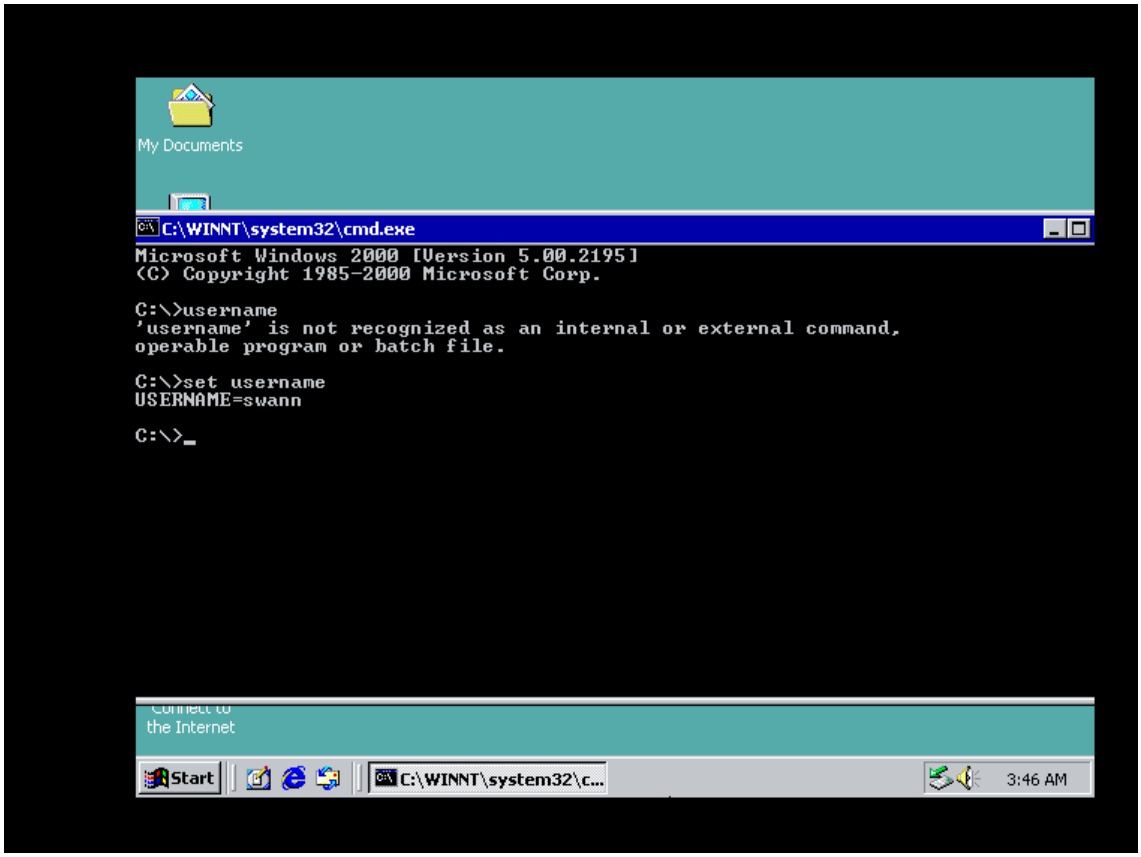


Figure 4.4.11: *Windows 2000 swann user*

On the Windows XP client the swann account triggers a login to the poisoned name. Because the XP client speaks to Responder's WebDAV/HTTP service, Responder captures a NetNTLMv1 challenge-response (an NTLMv1 hash) for swann from the XP machine (192.168.40.131). The older NetNTLMv1 is weaker and easier to crack than NetNTLMv2. This is a successful cross-machine capture: the victim and the target are now different hosts.

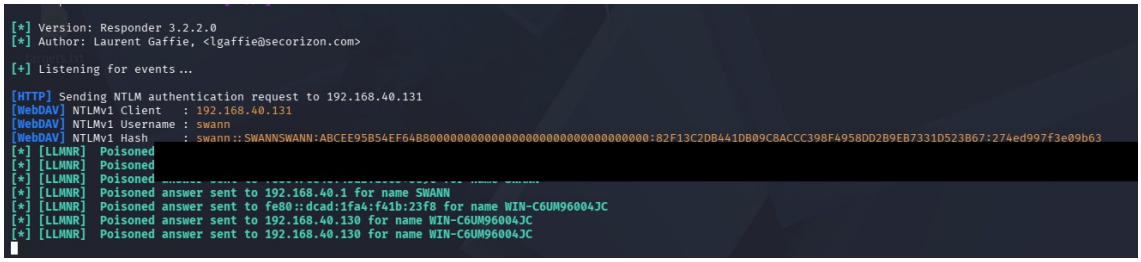


Figure 4.4.12: *NTLMv1 hash from XP client*

An attempt to relay this XP login over SMB hit a known tool bug: the XP client only speaks the old SMB1 protocol, and the relay server threw an error ("Client does not support SMB2"). Rather than fight the bug, the attack switched to the offline cracking path, which is just as valid and is often faster.

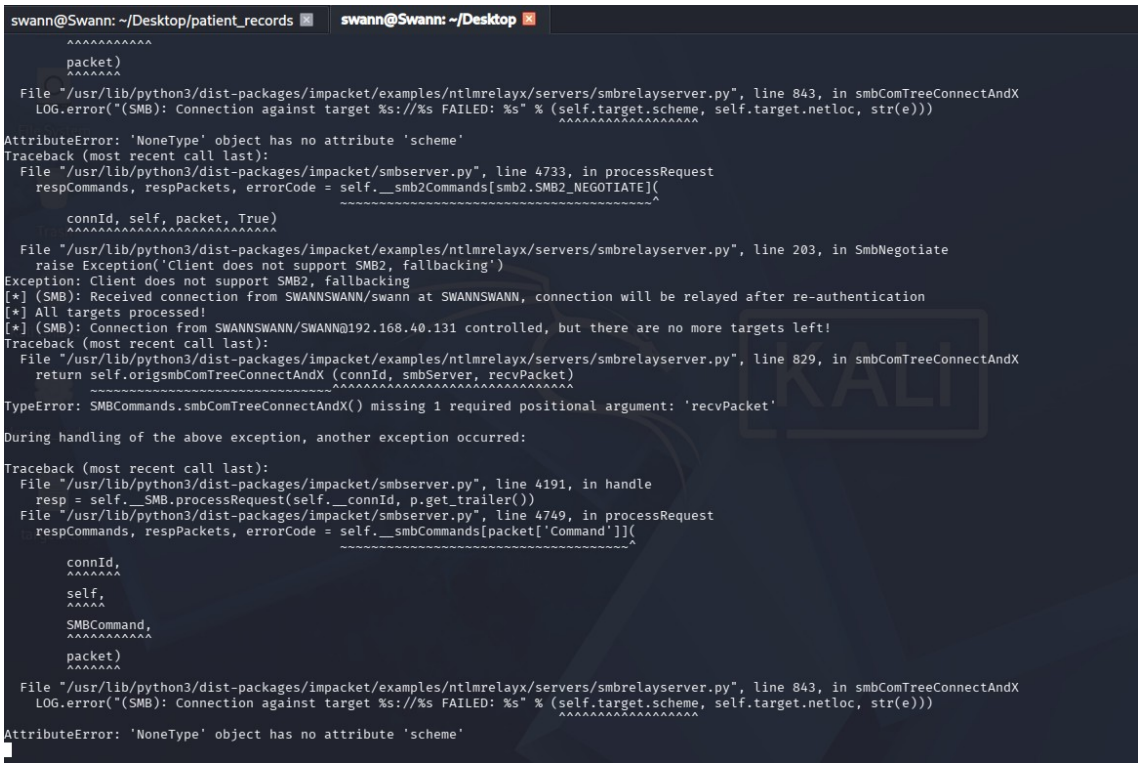


Figure 4.4.13: *SMB1 relay error*

The captured NTLMv1 hash was copied into its own file, and the rockyou wordlist was unzipped ready for cracking.



Figure 4.4.14: *Hash saved to file*



Figure 4.4.15: *Wordlist prepared*

hashcat was run in mode 5500 (NetNTLMv1) against the hash file using the rockyou wordlist. It cracked both hashes and recovered the plaintext password for the swann account. The status line reads Cracked and Recovered 2/2. The weak password fell in about one second, which shows how important strong passwords are.

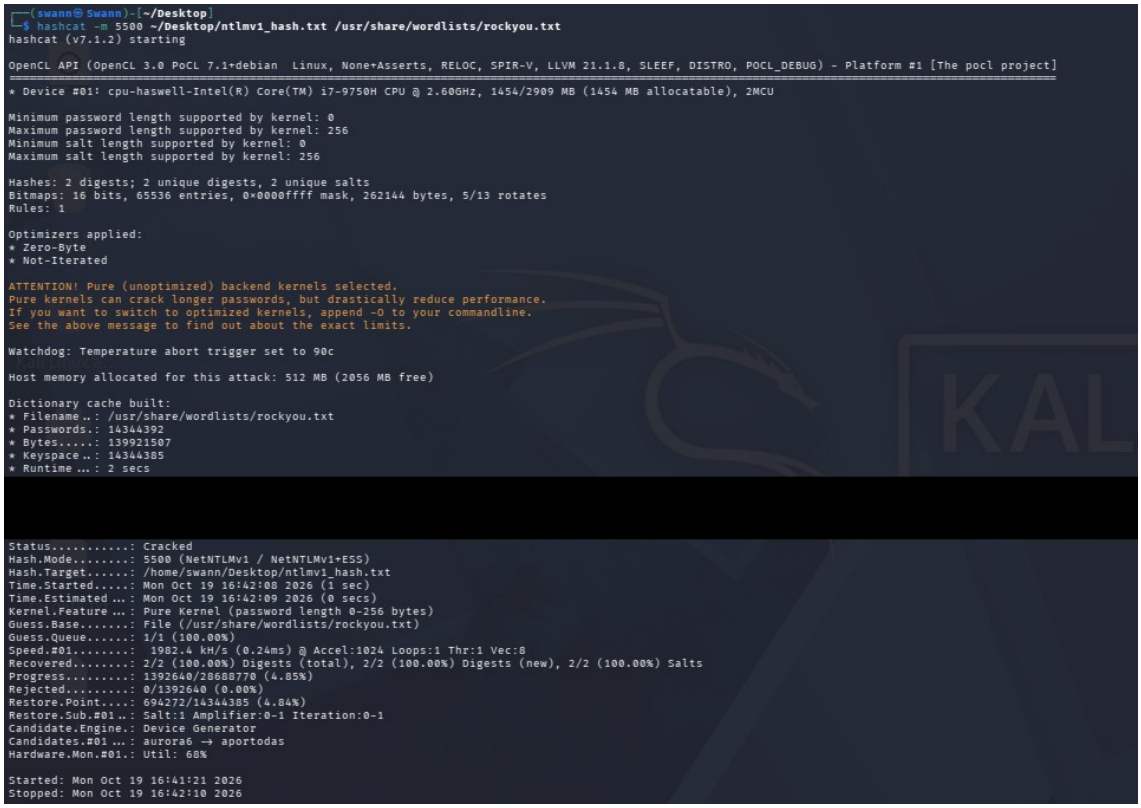


Figure 4.4.16: *hashcat password cracked*

At this point the attacker has moved from a captured challenge-response to a full working username and password. This is a credential compromise that gives unauthorized access as the staff user swann; it is not a rise to a higher privilege level such as administrator, which was not demonstrated in this test.

4.5 Phase 5 – Impact and Post-Exploitation

To prove real damage, the attacker uses the cracked login to reach the patient data. First smbclient was pointed at the XP client, which failed with a logon error. A check of XP's local security policy explained why: it was set to "Guest only – local users authenticate as Guest", a pre-existing default on that machine that forces network logins to the guest account. This was a finding, not a change – nothing was weakened. The attacker simply moved to the real target, the server.

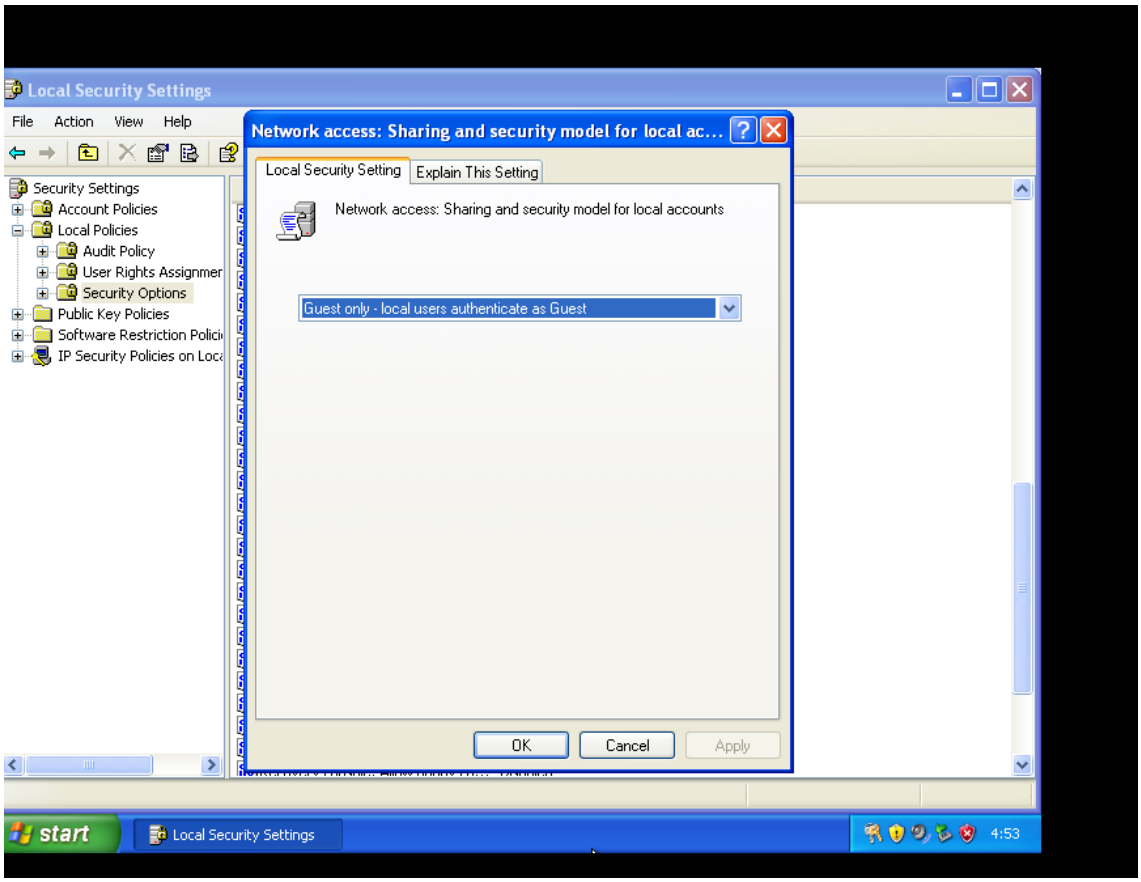


Figure 4.5.1: *XP Guest-only policy*

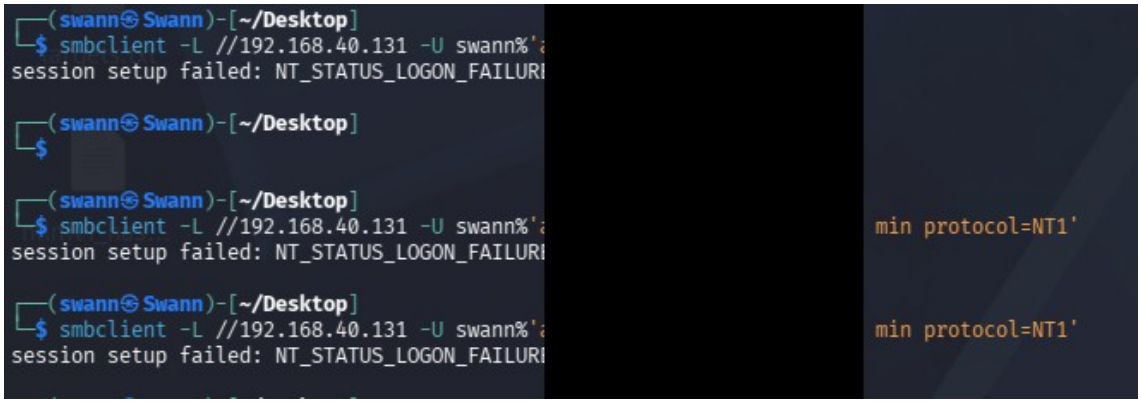


Figure 4.5.2: *smbclient to XP fails*

Using the cracked swann login, smbclient connects to the server's shares, opens the Users share, navigates to the Patient_Records folder and downloads patient_001.txt and patient_002.txt. This is the real impact: a person who started with no access has now copied private patient files off the hospital server.

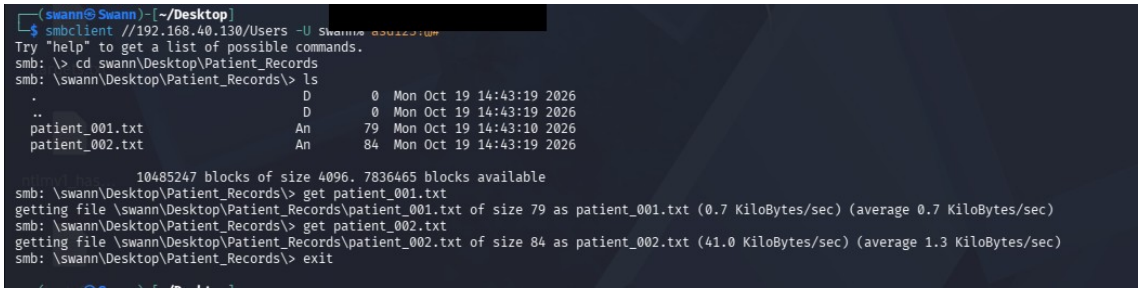


Figure 4.5.3: *Patient files downloaded*

In a real hospital this would be a serious data breach. Patient names, dates of birth, diagnoses and doctor notes would be exposed. This breaks patient privacy, breaks data-protection law, and could lead to blackmail, fraud or loss of trust in the hospital.

4.6 Phase 6 – Countermeasures and Mitigation

The best single fix for the relay part of this attack is to turn on SMB signing on the server. On the Windows Server, the policy "Microsoft network server: Digitally sign communications (always)" was set to Enabled.

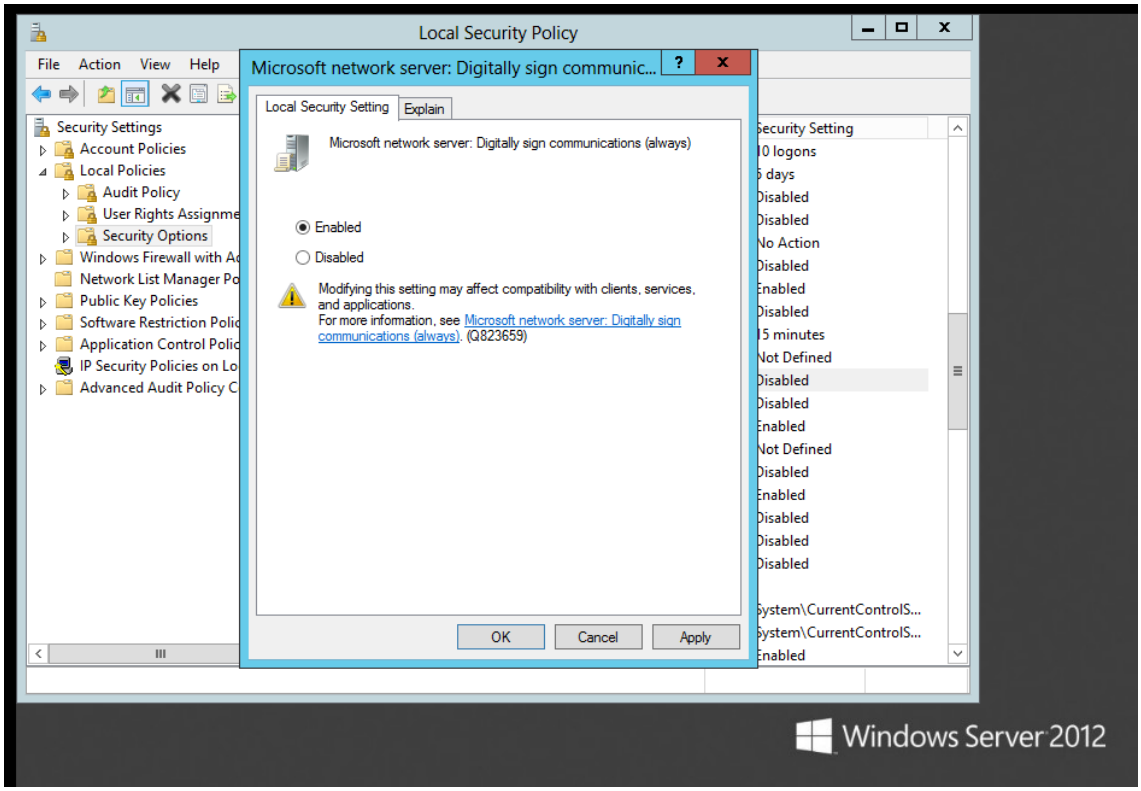


Figure 4.6.1: *SMB signing enabled (fix)*

A fresh nmap scan confirms the change at the network level: message_signing now reads required, and the smb2-security-mode script reports “Message signing enabled and required” (Figure 4.6.2). The same scan that earlier showed the weakness now shows the server is hardened.

```
┌──(swann㉿Swann)-[~/Desktop]
└─$ nmap -p 445 --script smb-security-mode,smb2-security-mode 192.168.40.130
Starting Nmap 7.991 ( https://nmap.org ) at 2026-10-20 15:30 -0400
Nmap scan report for 192.168.40.130
Host is up (0.0012s latency).

PORT    STATE SERVICE
445/tcp open  microsoft-ds
MAC Address: 00:0C:29:46:42:F2 (VMware)

Host script results:
| smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: required
| smb2-security-mode:
|   3.0:
|_    Message signing enabled and required

Nmap done: 1 IP address (1 host up) scanned in 6.81 seconds

┌──(swann㉿Swann)-[~/Desktop]
└─$
```

Figure 4.6.2: *Nmap re-scan after the fix – SMB signing now required*

After enabling signing, the relay attack was run again. This time ntlmrelayx failed with the message “Signing is required, attack won’t work”, and the login against the server was marked FAILED. The relay attempt that ran before is now blocked. This proves SMB signing stops the tested relay path; on its own it does not stop the capture-and-crack path shown earlier.

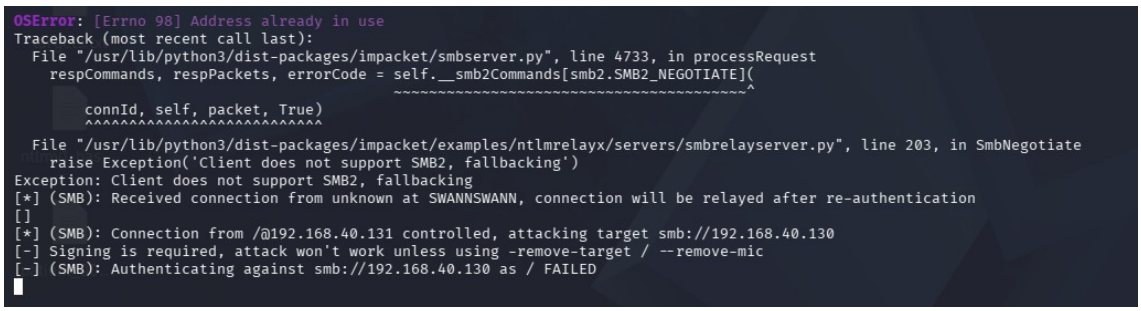


Figure 4.6.3: *Relay blocked after fix*

Other recommended fixes for a healthcare network:

• Turn off LLMNR and NBT-NS through Group Policy, so machines cannot be poisoned in the first place.

• Enforce strong, long passwords so that even a captured hash cannot be cracked from a wordlist.

• Use network segmentation so staff clients and servers that hold patient data are separated and watched.

• Monitor for Responder-style poisoning and for many failed SMB logins, so an attack is spotted early.

5. Critical Analysis and Impact

5.1 Critical analysis of the technique and tools

The LLMNR/NBT-NS poisoning technique is powerful but not perfect, and it is fair to look at both sides. Its big strength is that it is quiet and needs no password. It waits for normal network traffic instead of attacking a login screen, so it avoids account lockouts and most simple alarms. It also works on default Windows settings, so it needs no mistake by the user. Its main weakness is that it is passive: the attacker must wait for a victim to look up a name that cannot be resolved. If DNS always answers, or if no one makes a typo, nothing may be captured. It is also noisy on the wire – a defender who watches for poisoned LLMNR replies can spot it quickly.

Responder is reliable and easy to run, and it captures hashes well. But it is well known to defenders, and its poisoning can clash on the network if more than one tool answers at once. It

also needs its own SMB server turned off before a relay, as this test showed, or the two tools fight over the same port.

Impacket's ntlmrelayx is the strongest part of the chain when it works, because it needs no cracking at all. But it is fragile with very old clients. In this test it failed against the SMB1-only Windows XP machine because of a tool bug, and it could not relay a login back to the same host because Windows blocks that by design. These limits are important lessons: relaying needs a separate victim and target, modern-enough SMB, and SMB signing to be off. When any of those is missing, the attacker must fall back to cracking.

hashcat is very fast and cracked the weak password in about one second, but it is only as good as the wordlist and the password. A long, random password would not be in rockyou and would likely survive. So the cracking path depends entirely on weak passwords, which is itself a useful finding for the hospital.

Overall the technique is best seen as a chain of weak links rather than one magic exploit. It succeeds when several small defaults line up, and it is stopped by fixing any one of them. That makes it both a real threat and an easy problem to defend against, which is the key message for a healthcare IT team.

5.2 Impact to the organisation

This attack is dangerous because it needs no password to start and uses features that are on by default. The attacker never touches the login screen, so account lockout and most alarms do not fire. The only things needed are a foothold on the LAN and patience. That is a low bar for an insider or for malware that has already landed on one machine.

For a hospital the impact is high. The captured account, swann, was able to read the Patient_Records share. In the real world those files hold protected health information. A breach like this can mean legal fines under data-protection rules, harm to patients whose records leak, and lasting damage to the hospital's name. If the stolen account also had admin rights, the attacker could spread across the whole network, plant ransomware, or lock staff out of the systems they need to treat patients.

The root cause is weak default settings, not clever hacking. LLMNR/NBT-NS being on, SMB signing being off, and a weak password together formed a chain. Breaking any one link stops the attack, which is why the fixes above are cheap and effective. The test also showed that

defence in depth matters: even when SMB signing was off, the XP “Guest only” policy happened to block one path, and once signing was on the main relay path closed completely.

6. Conclusion

This demonstration showed a full attack chain against a healthcare file server, from an attacker with no access to a full data breach. By poisoning LLMNR and NBT-NS with Responder, the attacker captured a staff login; with hashcat the password was cracked offline; and with smbclient the patient records were downloaded. The weaknesses that made this possible were the legacy name services (LLMNR/NBT-NS) left switched on and a weak password; disabled SMB signing additionally exposed the server to relay attempts.

The tested relay path was then blocked by enabling SMB signing on the server, which made the relay attempt fail. SMB signing on its own does not close the whole chain, so it must be combined with turning off LLMNR/NBT-NS and enforcing strong passwords. Together these simple, no-cost configuration changes let the hospital defend itself, and the exercise met its aim of proving both the risk and the fixes in a safe, isolated lab.

References

Gaffie, L. (2024). Responder (Version 3.2.2.0) [Computer software]. GitHub. https://github.com/lgandx/Responder

Fortra. (2024). Impacket: Collection of Python classes for working with network protocols [Computer software]. GitHub. https://github.com/fortra/impacket

Lyon, G. (2024). Nmap: The network mapper [Computer software]. Nmap Project. https://nmap.org

hashcat. (2024). hashcat – advanced password recovery (Version 7.1.2) [Computer software]. https://hashcat.net/hashcat/

MITRE. (2024). Adversary-in-the-Middle: LLMNR/NBT-NS Poisoning and SMB Relay (T1557.001). MITRE ATT&CK. https://attack.mitre.org/techniques/T1557/001/

Microsoft. (2023). Overview of Server Message Block signing. Microsoft Learn. https://learn.microsoft.com/en-us/windows-server/storage/file-server/smb-signing

Microsoft. (2023). Network security: LAN Manager authentication level. Microsoft Learn. https://learn.microsoft.com/en-us/windows/security/threat-protection/security-policy-settings/